Introduction

Amdhal law:theoretical speedup in latency of the execution of a task at fixed workload.

Amdahl' s Law

- f: Parallelizable fraction of a program
- N: Number of processors

$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{N}}$$

Serial bottleneck of Amdahl' s Law: Maximum speedup (1/(1-f)) limited by serial portion (1 - f)

Parallel portion (f) is usually not perfectly parallel: Synchronization overhead (e.g., updates to shared data), Load imbalance overhead (imperfect parallelization) ,Resource sharing (contention among processors)

Roofline model:Theoretical performance bound of your application running on your machine.

Computing regime: Latency-limited -> throughput-limited

Roofline Model's Two Perspectives:

1, Target processor's perspective：Showing inherent hardware limitations (or bound), in term of compute and memory

2, Compute kernel's perspective： Showing the priority of optimizations for a given compute kernel running on a given processor

Arithmetic intensity (AI) :AI = Total Flops / Total Memory Bytes ,describes the characteristics of a compute kernel running on a given processor

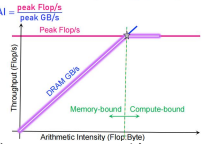
Large AI - Compute-bound ,Small AI - Memory-bound

Roofline model's 3 Steps:

1, Machine characterization: Memory bandwidth, Peak compute; 2, Application Characterization: Arithmetic intensity; 3, Application execution monitoring: Real Throughput

Attainable Flop/s = min(peak Flop/s, AI * peak GB/s)

☆ AI = peak Flop/s / peak GB/s



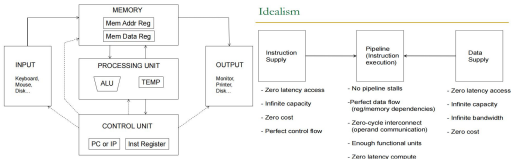
斜率:cache>HBM>DEAM,平台：peak flops>vec>no vectorization

Little ' s Law: L = λ *W (buffer size = throughput * latency)

The long-term average number L of customers in a stationary system is equal to the long-term average effective arrival rate λ multiplied by the average time W that a customer spends in the system.

Von Neumann model

Components: memory(stores the program and data), processing unit, input, output, control unit(controls the order in which instructions are carried out)



Idealism

Memory stores programs and data, bit logically grouped into bytes and words(8/16/32 bits)

Address space: total number of uniquely identifiable locations in memory.

Addressability:how many bits are stored in each location/address (8-bit(byte) /word-addressable)

PU(processing unit) performs the actual computations, consists of ALU and temporary storage(regfile)

ALU execute computation and logic operations, combine a variety of arithmetic and logic operation into a single unit.perform only one function at a time.

Registers:ensure fast access to values to be processed in ALU, one reg contain one word.

Register set/file:set of registers manipulated by instructions.MIPS 32 general purpose registers.

Input and output enables information to get into and out of a computer.

Control unit:conductor of an orchestra.conduct execution,track instruction being process(IR) and to be processed(PC &IP instruction pointer).

Key properties of Von Neumann Model: stored program, sequential instruction processing.

ISA: interface between software command and hardware carry-out.specify memory organization(address space/ addressability ,word/byte addressable) register set(32 in MIPS), instruction set(opcodes, datatype, addressing model(immediate or literal, register, memory addressing modes(PC-relative, pseudo-direct, base+offset)).more addressing modes create more work for compiler and microarchitect.

ABI: application binary interface, between two binary program modules.

Semantic gap:语义鸿沟，存在于高层次语义和低层次操作之间。

Opcode: what the instruction does;

Operands: who the instruction is to do with.source/destination ~

Type of opcodes: operate(R/I/F), data movement, control(conditional branches and unconditional jumps)

Complex instructions:+denser encoding ,+simpler compiler, -larger chunks of work ,-complex hardware

Large number of registers: better register allocation but larger instruction size and larger register file size.

Instruction cycle: the process of executing an instruction.

IF-ID-EXE-MEM-WB. LDR no EXE, R/I no MEM,SW no WB, branch no MEM,WB, jump no ID,EX,MEM,WB

Single-cycle CPU:AS(architecture state)-process-AS'(state,next-state logic), combinational logic, 无中间状态

Microarchitecture implement how AS transformed to AS'.

MIPS state element: PC, instruction memory,register file, data memory.

Clock time determined by slowest instruction.

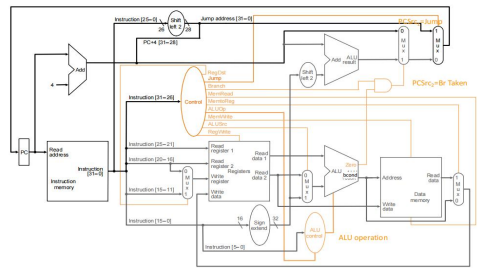
Multi-cycle CPU

Decrease clock cycle time; each instruction take as many clock cycle as it needs to take.

AS->AS+MS1->AS+MS2->AS+MS3->AS'

Benefit:reduce critical path; bread and butter design（优化常用指令）; balanced design（硬件资源复用）

Execution time: CPI(>1) * clock cycle time(short)



Pipeline CPU: more concurrency and higher instruction throughput(1/CPI).

Ideal assumption: repetition of identical/independent operations; uniformly partitionable suboperations.

Lec2: pipeline hazard+reorder buffer

Pipeline hazards: prevent instruction from executing next stage.

Structural hazards:limited hardware resources->replicating hardware resources.

Data hazards: flow dependence(RAW, true 依赖), output dependence(WAW), anti dependence(WAR)

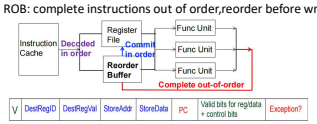
RAW: hardware pipeline stall; software pipeline stall(nops); data forward/bypass

Control hazard: data dependence on IP/PC.

Reorder buffer

Exceptions: internal problems,handle when detected. Interrupts: external events,handle when convenient.

ROB: complete instructions out of order,reorder before writing result to AS.



V	DestRegID	DestRegVal	StoreAddr	StoreData	PC	Valid bits for reg/data + control bits	Exception?

ROBs need to:

1, correctly reorder instructions back into the program order

2, update the architectural state with the instruction' s result(s), if instruction can retire without any issues

3, handle exception/interrupt precisely, if needs to be handled before retiring the instruction

4, use valid bits to keep track of readiness of the result, find out if the instruction has completed execution

Lec3: Tomasula

Solution: out-of-order dispatch

Reservation station: move dependent instructions out of the way of independent ones.instructions dispatched in dataflow order. benefit: latency tolerance.

Two humps in modern pipeling: reservation stations(enable in-order issue and out-of-order dispatch), reorder buffer(enable OoO completion, in-order commitment)

1. register renaming; 2. insert into reservation station; 3.broadcast tag when value produced and compare; 4.all values ready, dispatch into FU(functional unit).

tomasula 三大组件：寄存器重命名表(RAT,register alias table), RS，公共数据总线(CDB)

Dataflow graph limited to instruction window(all decoded not retired instructions)

Superscalar+SIMD+multi-core

Superscalar execution: N-wide superscalar->N instructions per cycle

Need to add hardware;hardware check dependence between concurrently-fetched instruction. +higher throughput -higher complexity for dependence checking -more hardware needed

Vector Insn

SISD: Single instruction operates on single data element

SIMD: Single instruction operates on multiple data elements (Array processor ,Vector processor)

MISD: Multiple instructions operate on single data element (Closest form: systolic array processor, streaming processor)

MIMD: Multiple instructions operate on multiple data elements (multiple instruction streams) (Multiprocessor ,Multithreaded processor)

For SIMD,Memory (bandwidth) can easily become a bottleneck, especially if compute/memory operation balance is not maintained or data is not mapped appropriately to memory banks

增加向量寄存器堆，multiple bank（内存分为多个块）

scalar 和 SIMD 提高 peak flops

Fine-grained multithreading：Hardware has multiple thread contexts (PC+registers). Each cycle, fetch engine fetches from a different thread.

无需检查指令间依赖和分支预测逻辑，提高吞吐量：硬件复杂度高，单线程性能下降；资源竞争：仍需依赖检查，如访存。

可以使实际性能上移，更接近理论峰值。增加带宽压力，cache 命中率下降

Multi-core processors:Put multiple cores on the same die.

Alternative: bigger, more powerful single core：对程序员透明，提高单线程性能；功耗高，收益递减，设计复杂。

多核：简单，能效高，频率容易提升，吞吐量高；需要并行编程；单线程性能低；引脚带宽限制。

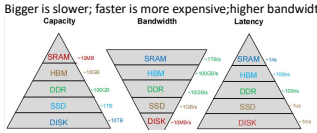
Technology push: instruction issue queue; large, multi-ported register files

成倍提高 peak flops，加剧方寸受限

Memory

Property of ideal memory: zero latency, infinite capacity, infinite bandwidth, zero cost

Bigger is slower; faster is more expensive;higher bandwidth is more expensive.



Flip-Flops (~K) :Very fast, parallel access, Very expensive (one bit costs tens of transistors)

Static RAM (~M) :Relatively fast,one data word at a time ,Expensive

Dynamic RAM (~G) :Slow, one data word at a time, reading destroys content (refresh), needs special process for manufacturing ;Cheap (one bit costs only one transistor plus one capacitor)

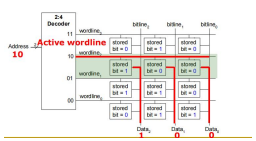
Flash Memory (~T) :Much slower, access takes a long time, non-volatile ;Very cheap

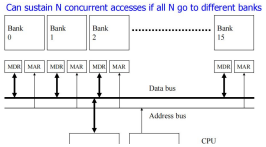
Array organization of memory: to efficiently store large amounts of data: memory array(store data),address selection logic(one row of array), readout circuitry(read data out)

Two-dimensional array of bit cells: each bit cell store one bit.N address bits and M data bits.2^n rows, M columns.depth:number of rows(number of words), width:number of cols(size of word), array size: depth * width=2^n * M

Bitline: Storage nodes in one column connected to one bitline

Wordline: Address decoder activates only one wordline, content of one line of storage available at output





SRAM : a bit of static random access memory

Access transistors connect the bit storage to the bitline; access controlled by the wordline

Goal: buffering data on chip to reduce external memory traffic, high performance ,low capacity. Used in cache in CPU, shared memory in GPU, in-chip buffer in AI accelerator.

Memory banking: memory divided into banks that can be accessed independently; share address and data buses; start and complete one bank access per cycle.

Read Sequence: 1. address decode 2. drive row select 3. selected bit-cells drive bitlines (entire row is read together) 4. differential sensing and column select (data is ready) 5. precharge all bitlines (for next read or write)

Access latency dominated by steps 2 and 3 . Cycling time dominated by steps 2, 3 and 5

step 2 proportional to 2^m, step 3 and 5 proportional to 2^n

DRAM

Key components of computing system: computation, communication, storage/memory

Computation is bottlenecked by memory(important workloads data intensive, data increasing)

Performance/energy/reliability perspective: memory as bottleneck.

High level abstraction: map virtual memory addresses to physical memory

Memory optimization aim at size, bandwidth, not latency.

DRAM: dynamic random access memory

Capacitor(电容器) charge state indicated stored value.capacitor leak through RC path.DRAM cell lose charge overtime and need to be refreshed.

Large memories are hierarchical array structures.

DRAM: channel->rank->bank->subarrays->mats./channel-DIMM-rank-chip-bank-row/col

Interleaving/banking: reduce latency of memory array access,enable multiple access in parallel. Divide a large array into multiple banks. Access to different banks can be overlapped;

Performance characteristics of memory

Address bits: SRAM in the same chip with compute units,relatively small;

DRAM separate chips from compute units, pin number bottleneck,large number of address bits

DRAM Read Sequence: 1. address decode 2. drive row select 3. selected bit-cells drive bitlines 4. a flip-flopping sense amp amplifies and regenerates the bitline, data bit is mux' ed out 5. precharge all bitlines. Destructive reads, charge loss over time.

A DRAM controller must periodically read each row within the allowed refresh time.

DRAM access state:page hit(memory transaction access a row opened in bank,no precharge or activate), page closed(access a row whose corresponding bank is closed, activate required), page miss(access a row not matching the active row in bank, 1 precharge, 1 activate)

Take away message: sequential>random(pre-charge and activate needed)

refreshing: -energy consumption, -performance degradation, -QoS/predictability impact.

Manufacturing process variation: DRAM cells not exactly the same, some leakier.

HBM +high bandwidth, +low power consumption, -less flexibility, -low capacity, -high cost

SSD

Phase change material exists in amorphous(low optical reflexivity, high electrical resistivity) and crystalline(high~, low~). PCM is resistive memory: high resistance(0), low~(1)

Graphics processing units

CPU: few complex cores, large cache, large and slow memory

GPU:lots of simple cores, small cache, small and fast memory

GPU computing: computation offloaded to GPU. CPU-GPU data transfer(1)->GPU kernel execution(2)->GPU-CPU data transfer(3)

GPU are SIMD engines underneath, yet programming done using threads

Programming model(software):how programmer express code,like data parallel, dataflow,...

Hardware execution model: how hardware execute the code, OoO, vector processor,...

Programming model

Sequential(SISD):executed on:piplined, OoO execution, superscalar/VLIW processor

Data Parallel(SIMD): generate SIMD instruction

Multithread(SPMD, single program multiple data): generate thread to execute each iteration.

SPMD: multiple instruction stream execute same program, different data and control-flow path

CUDA(SPMD example):grid, thread block, thread

CUDA program structure:

1.Function prototypes:float serialFunction(…); __global__ void kernel(…);

2.main()

Memory allocation :cudaMalloc((void**)&d_in, #bytes);

Memory copy :cudaMemcpy(d_in, h_in, #bytes, cudaMemcpyHostToDevice);

Kernel launch :kernel<<< #blocks, #threads >>>(args);

Memory deallocation :cudaFree(d_in);

Explicit synchronization :cudaDeviceSynchronize();

3.Kernel- __global__ void kernel(type args,…)

blockDim: block dimension ,blockIdx: block id within grid , threadIdx: thread index within block

SIMT(hardware) & warp(software)

SIMT:single instruction multiple thread; warp: basic execution unit with 32 consecutive threads, thread block divided into warps for SIMT execution.can reduce GPU scheduling overhead

SIMT: + treat each thread separately, +group threads into warps flexibly

GPU optimization

GPU memory: optimize with multithreading, memory coalescing, shared memory

FGMT hide long latency operations, occupancy=active warps/maximum number of warps

Memory coalescing: thread in same warp access consecutive memory location in same burst, combine and serve by one burst.make sure that concurrent threads access nearby memory location. Peak bandwidth utilization when all threads in warp access one cache line.

Memory divergence: location not in same burst, multiple transactions needed, longer time

Shared memory: interleaved/banked memory, bank=address%12(NVIDIA)

Bank conflicts only possible within a warp. Due to strided access.

Optimization: padding, randomized mapping, hash functions

Data reuse: tiling: divide input into tiles, load L_size chunks into shared memory, compute. Loading/compute amount: (L_size+2)^2/L_size^2 per thread.

SIMT efficiency

Branch divergence occur when threads inside warps branch to different execution paths.

Atomic operation: on shared/global memory,perform read-modify-write operation atomically.

Atomic conflict=serialized updates, prevent data races when two thread update same location.

Estimates $t_E + \frac{t_T}{\#Streams}$ $t_T + \frac{t_E}{\#Streams}$

$t_E >= t_T$ (dominant kernel) $t_T > t_E$ (dominant transfers)

Dynamic warp formation/merging: merge threads executing same instruction.

Generic Term	NVIDIA Term	AMD Term	Comments
Vector length	Warp size	Wavefront size	Number of threads that run in parallel (lock-step) on a SIMD functional unit
Pipelined functional unit / Scalar pipeline	Streaming processor / CUDA core	-	Functional unit that executes instructions for one GPU thread
SIMD functional unit / SIMD pipeline	Group of N streaming processors (e.g., N=8 in GTX 285, N=16 in Fermi)	Vector ALU	SIMD functional unit that executes instructions for an entire warp
GPU core	Streaming multiprocessor	Compute unit	It contains one or more warp schedulers and one or several SIMD pipelines

Cache:Have multiple levels of storage, most data processor needs kept in faster levels.

Memory hierarchy:regfile-> L1 cache-> L2 cache->L3 cache->main memory->swap disk

Cache feasibility: temporal locality, spatial locality

Recently accessed data will be again accessed in the near future.

Nearby memory locations will be accessed soon.

$T_i = hr_i \cdot t_i + mr_i \cdot (t_i + T_{i+1})$

$T_i = t_i + mr_i \cdot T_{i+1}$ r_i: hit rate, mr_i: miss rate

Cache: an automatically-managed memory structure

Blocks: main memory logically divided into fixed-size chunks.cache house limited number of it. Each block address maps to a potential location in cache.

Cache hit rate = hits/(hits+misses)=hits/accesses

AMAT(average memory access time)=(hit-rate*hit-latency)+(miss-rate*miss-latency)

Cache access: 1) index into the tag and data stores with index bits in address; 2) checks valid bit in tag store; 3) compares tag bits in address with the stored tag in tag store; 4) If a block is in the cache (cache hit), the stored tag should be valid and match the tag of the block, and read out data.

Cache block(line):unit of storage in cache.

Placement: where and how to place/find a block in cache?

Directed-mapped: a chunk go to only one cache block.+easy to implement.-low hit rate

Fully associative: a chunk go to any cache block.+highly utilization.-difficult to implement.

Set-associative: a chunk go to N cache blocks in N-way set-associative cache.+accommodate conflict better.more complex, slower access, larger tag store.

Degree of associativity: how many blocks can map to the same index.high associativity->high hit rate, slow cache access time, expensive hardware.

Replacement: what data to remove to make room in cache?

Any invalid block first. If all valid:random, FIFO, least recently used, hybrid ~, optimal~

LRU: need to keep track of access ordering of blocks.use approximation.

Pseudo LRU:PLRU replacement way selection, PLRU bits updating rule.

Set thrashing: program working set larger than set associativity, random better.

Belady's OPT:replace the block going to be referenced furthest.

Granularity of management: large or small blocks? Subblocks?

Write policy: what do we do about writes?

1. store insn->cache:write-allocate policy(memory, default, allocate the cache line), write-no-allocate policy(PCIe/IO,write directly to memory without allocation in cache)
2. Cache->memory: write-back policy(default, write to cache until cache kick cache block out), write-through policy(streaming write instruction, write to cache and memory right away)

Instructions/data: do we treat them separately?

Unified cache:+dynamic sharing of cache space.-instructions and data thrash each other.-I and D accessed in different places in pipeline.

Cache miss:compulsory miss(first reference, prefetching)

capacity miss(cache too small,utilize space, software management)

conflict miss(other,more associativity(victim cache, better, randomized indexing))

Improve cache performance: reduce miss rate, reduce latency/cost, reduce hit latency/cost

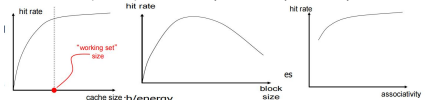
Capacity (C): the number of data bytes a cache stores

Block size (b): bytes of data brought into cache at once

Number of blocks (B = C/b): number of blocks in cache: B = C/b

Degree of associativity (N): number of blocks in a set

Number of sets (S = B/N): each memory address maps to exactly one cache set



Cache in multi-core CPU

Private cache:cache belong to one core.

Shared cache: cache shared by multiple cores.

Resource sharing: allow multiple contexts to use hardware resource.+improve utilization/efficiency(throughput);+reduce communication latency;+compatible with shared memory programming model;-reduce some thread's performance;-eliminate performance isolation

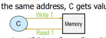
Shared cache between cores: +high effective capacity; +dynamic partitioning of available cache space;+easier to maintain coherence; -slower access; -conflict miss; -hard to guarantee a minimum level of service.

Coherence: ordering of operations from different processors to the same memory location.

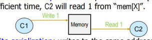
Consistency: ordering of all memory operations from different processors.

Cache Coherence: Multiple cores have a consistent state of the last written value from any core to a memory address.

- Program order preservation: core C writes to the address and then reads from the same address, C gets value written.



- Coherent memory view: if C1 performs "mem[X] = 1", after a sufficient time, C2 will read 1 from "mem[X]".



- Write serialization: writes to the same address by different processors are seen in same order by all processors.

Consistency: A contract between programmer and microarchitect.

- Preserving an "expected" (more accurately, "agreed upon") order simplifies programmer's life
  - Ease of debugging; ease of state recovery, exception handling
- Preserving an "expected" order usually makes the hardware designer's life difficult
  - Especially if the goal is to design a high performance processor: Recall load-store queues in out of order execution and their complexity

Hardware architecture for cache coherence: cache tags(MESI),cache updating(invl./uodate), interconnect(bus/switch)

Bus-based protocol(one trans a time):

1, A cache arbitrates for bus access, waiting until 2 happens

2, A cache is granted bus access

3, A cache places command on bus, waiting until 4 happens

4, Other caches place responses on bus

Switch-based protocol: Each core pair can independently communicate with each other.

Cache Updating policy: update replicated data in other caches.

Update protocol:Broadcast written data and address to cores ;Cores update the data in their caches if block is present

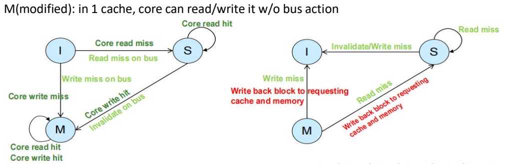
Invalidate protocol: Broadcast invalidation of address to sharers ;Cores invalidate block in their caches if block is present

MSI protocol: safely replicated data in caches.

I(invalid): block is not in cache, need to fetch from memory or other cache

S(shared): in >=1 caches, local cores can read it w/o bus action

M(modified): in 1 cache, core can read/write it w/o bus action



MESI Protocol: Illinois protocol (ISCA, 84)

- I(nvalid): block is not in cache, need to fetch from memory or other cache
- S(hared): in >1 caches, clean, local cores directly reads it w/o bus action
- M(odified): in 1 cache, local core can read/write it w/o bus action
- E(xclusive): in 1 cache, clean, local core reads/writes it w/o bus action

MSI:

Time	P1 op.	P2 op.	State A in P1	State B in P2	Bus action
t0			I	I	
t1	Read A		S	I	Read miss A
t2	Write A		M	I	Invalidate
t3		Read B	M	S	Read miss B
t4		Write B	M	M	Invalidate

Key Differences from MSI Protocol:

- Local core reads block in state E, the state holds
- Local core writes block in state E → state M, without bus action
- Remote core reads, via read miss on bus, block in state E → state S
- Remote core writes, via write miss on bus, block in state E → state I

MESI:

Time	P1 op.	P2 op.	State A in P1	State B in P2	Bus action
t0			I	I	
t1	Read A		E	I	Read miss A
t2	Write A		M	I	
t3		Read B	M	E	Read miss B
t4		Write B	M	M	

Snoop-based cache coherence:Single point of serialization for all memory requests; One outstanding memory request per processor; System interconnect is an atomic shared bus (one cache communicates at a time)

Bus: ordering, communication(simple, fast broadcast medium)

Directory-based cache coherence:Address the lack of scalability of snooping

Logically-central directory keeps track of where copies reside. Cache consult directory.

Coherence still require single point of serialization; serialization location different for blocks.

Home Node: owns the corresponding directory, each cache block have different home node.

Local Node: the node initiates the cache read/write requests

Remote Node: the node passively responses to the action from the home node

Local node:

- **On a bus read**: send out *getS* request to directory node, after receiving the data, set the cache's bit.
- **On a bus write**: send out *getEx* request to directory node
  - invalidate all caches that have the block and reset their bits
  - Have an "exclusive bit" associated with each block in each cache

2-bit	log_2N-bit	N-bit
states	Owner	Sharer list (one-hot bit vector)

Directory node:

- 1, Receives *getS, getEx* requests from nodes
- 2, Based on different cache states:
  - 2a, Sends *Invalidate* messages to sharers if needed
  - 2b, Forwards request to memory if needed
- 3, Replies to requestor and updates sharing state

Detailed directory for each cache line:

- Each cache block needs $N+log_2N + 2$ bits for its directory, which resides at the home node.
  - 2-bit cache states: a block is owned by the directory unless the block is in a cache in state M. State M means a node writes to it.
  - One shared bit for each cache: indicating whether the block is shared in a cache
  - log_2N owner bit: indicates that the cache that has the only copy of the block and can update it without notifying others

Memory ordering in MIMD processor

Load-Load: Effectively prevents ordering of loads performed before the barrier with loads performed after the barrier

Load-Store: :Effectively prevents ordering of loads performed before the barrier with writes performed after the barrier

Store-Store: Effectively prevents ordering of stores performed before the barrier with stores performed after the barrier

Store-Load: Effectively prevents ordering of stores performed before the barrier with loads performed after the barrier

Consistency model

Sequential consistency:memory is a switch that services one load or store at a time.

Total store order: SC+store buffer. unable with store-store/load

Partial store order:load-load/store using total store order, rest using write coalescing

Protecting shared data: threads can't update shared data concurrently; access to shared data encapsulated inside critical sections or protected via synchronization constructs;mutual exclusion principle: one thread execute a critical section at a time; correct execution.

First-level caches (instruction and data)

- Decisions very much affected by cycle time
- Small, lower associativity; latency is critical
- Tag store and data store usually accessed in parallel

Second-level caches

- Decisions need to balance hit rate and access latency
- Usually large and highly associative; latency not as important
- Tag store and data store can be accessed serially

Serial vs. Parallel access of levels

- Serial: Second level cache accessed only if first-level misses
- Second level does not see the same accesses as the first
  - First level acts as a filter (filters some temporal and spatial locality)
  - Management policies are therefore different

深度学习加速器

Five Design Principles:

- **Global Buffer:** 使用专有的存储器来减少数据搬运的距离与开销，比如将复杂的cache设计替换成scratchpad memory (global buffer)。
- **简化控制模块:** 将缩减的高级微架构特性而节省出的面积，用于增加更多的运算单元或者片上存储。
- **并行计算模块:** 使用能够符合特定领域加速需求最简单的并行形式，例如：对于矩阵运算的加速，单条指令直接支持小矩阵运算。
- **量化:** 减少计算数据尺寸与类型来符合特定领域性能要求，例如，深度学习中，推理可以采用int8量化方式进行。
- **专用编程语言:** 使用DSA专用语言进行编程。

	CPU	DSA
On-chip Memory	Cache	Global Buffer
Instruction Issue	Superscalar	In-order/simple
Parallelism	Inter-instruction	Intra-instruction
Fuctionality	Full	Partial
Optimization Purpose	Low Latency	High Throughput
Programming Language	General	Domain-specific

Operator	计算特性	访存特性
Conv	矩阵相乘	Burst+stride
Activation	单向量操作	Sequential
Pooling	单矩阵Reduce操作	Burst+stride
FC	矩阵相乘	Sequential

AI processors

Two main properties of AI accelerator: 计算，访存

目标：减少内存访问、减少 global buffer 访问，增加计算

Cache-based Programming Model: Automatic

Buffer-based ~: manual manipulation. Global buffer:separate, low unit access cost

WS(weight stationary):减少从 global buffer 读取 weight,广播 activation,PE 水平累加 Psum

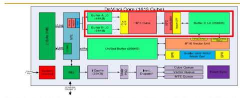
OS(output stationary):减少从 global buffer 读取存储 Psum,广播 weight,PE 水平复用 activation

IS(input stationary):减少从 global buffer 读取 activation,并行读 weight,水平累加 Psum

RS(row stationary):global buffer 读 filter 一行,activation 一滑窗，减少从 global buffer 整体读出

增加计算模块：Cube（单指令处理小矩阵乘法）, Vector（单指令处理向量操作）

Recall: Ascend Cube模块（算力担当）



矩阵乘运算单元Cube：一拍完成一个fp16的 2个16x16矩阵相乘；C = A * B；如果是int8输入,则一拍完成 16*32 与 32*16 矩阵乘。

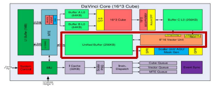
累加器Accumulator: 把当前矩阵乘的结果与前次计算的中间结果相加（C = A * B + C），可以用于完成卷积中加bias操作。

L0A/L0B/L0C Buffer：L0A 存储矩阵乘的左矩阵数据，L0B 存储矩阵乘的右矩阵数据，L0C 存储矩阵乘的结果和中间结果。

A/B DFF: 数据寄存器，缓存当前计算的16*16 左/右子矩阵。

Accum DFF：数据寄存器，缓存当前计算的16*16结果矩阵。

Recall: Vector模块（多面手）



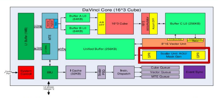
向量运算单元Vector Unit：覆盖各种基本的计算类型和许多定制的计算类型，主要包括FP16/FP32/int32/Int8等数据类型的计算，支持连续或者固定间隔寻址；或者VA寄存器寻址（不规则向量运算）

SIMD长度：一条Vector指令可以完成两个128长度fp16类型的向量相加/乘，或者64个fp32/int32类型的向量相加/乘

Unified Buffer(UB)：保存Vector运算的源操作数和目的操作数；一般要求32Byte对齐；

数据从L0C->UB：随数据搬运在Vector Unit完成一些RELU/数据格式转换等操作

Recall: Scalar模块（司令部）



- **Scalar Unit**：负责完成AICore中的标量运算，功能上可以看做一个小CPU；完成整个程序的循环控制、分支判断、CUBE/Vector等指令的地址和参数计算以及基本的算术运算等
- **Unified Buffer or Scalar Buffer**：昇腾310/910 Scalar Unit不能直接访问外面的DDR/HBM，需要预留UB的一部分(310)或者使用专门的Scalar Buffer(910)用作Scalar Unit的堆栈空间
- **GPR**：通用寄存器，目前包含32个通用寄存器
- **SPR**：专用寄存器，为了支持指令集一些指令的特殊需要，Davinci设计了许多专用寄存器，比如CoreID, BLOCKID, VA, STATUS, CTRL等寄存器

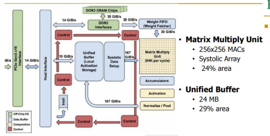
Davinci架构的优势：

- CUBE极致算力高 —— 同等功耗和面积下，Davinci Core比Nvidia V100/TPU 极致算力都高；功耗面积相似的情况下，昇腾910算力是Nvidia V100 2.1倍
- Buffer访问、管理效率高：单DavinciCore内 CUBE/VECTOR/MTE 有效并行+丰富的片上Buffer和带宽，让Davinci 能够高效的发挥极致算力，且有效控制功耗
- 硬核随路计算指令：提供了硬件支持的Img2Col/格式转换等随路计算指令，方便了程序设计

Davinci架构的不足：

- 难编程：对编程人员要求比较高（事件同步、Buffer使用），编程易用性有待提升
- 生态不完善：软件生态才开始，相关配套工具、包括Debug手段、PMU等都还不够丰富

Recall: TPU v1

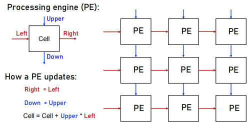


- **Matrix Multiply Unit**
  - 256x256 MACs
  - Systolic Array
  - 24% area
- **Unified Buffer**
  - 24 MB
  - 29% area
- **TPU v1**
  - For inference, model is pre-stored in DDR3, and data is from the host via PCIe

Recall: Systolic Arrays in AI Accelerator

- Systolic array can be multi-dimensional
  - The most popular one used by AI accelerator is two-dimensional

Processing engine (PE):



How a PE updates:

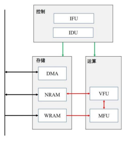
Right = Left

Down = Upper

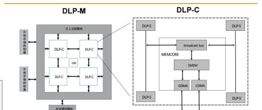
Cell = Cell + Upper * Left

Overall Architecture of DLP-S

- Control Module
  - IFU (Instruction Fetch Unit)
  - IDU (Instruction Decode Unit)
- Compute Unit
  - VFU (Vector Function Unit)
  - MFU (Matrix Function Unit)
- SRAM Unit
  - WRAM (Weight RAM)
  - NRAM (Neuron RAM)
  - DMA (Direct Memory Access)



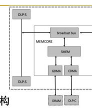
DLP-M总体架构



- 多核处理器分层结构设计
  - 一个DLP-M由多个DLP-C构成
  - 一个DLP-C由多个DLP-S构成

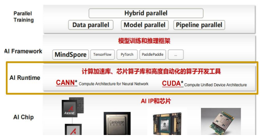
为什么需要进行分层结构设计？ 减少NoC的负载核开销

DLP-C总体架构



- **DLP-C整体架构**
  - 四个DLP-S
  - 存储核MEMCORE（Memory Core）
    - 存储SMEM：DLP-S共享数据
    - 通信：
      - GDMA: DLP-C与片外DRAM
      - CDMA: DLP-C之间，多个DLP-S之间

AI Architecture



The motivation of NN operator library:

1, NN tasks are composed of NN operators

2, AI chips are difficult to program, we cannot let AI programmer directly program AI chips

The goal of NN operator library:

Performance + Usability: provide high-performance, well-documented NN library for the upper AI framework such as MindSpore.

Parallel training

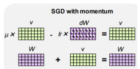
AI system: four components of model training: computing, storage, network, compiling

Forward pass:loss function:indicating how wrong the network was; goal: minimize loss value

Backward pass:goal:compute gradients to layer weights.

- Weight update (SGD)
  - Input: Weight W, gradient dW
  - Output: updated weight
  - Operation:
    - Increment each weight with the corresponding gradient value

- Weight update (Momentum)
  - Input: Weight W, gradient dW
  - States: 1 momenta (~model)
  - Output: updated weight
  - Operation:
    - Update internal state with weight gradient, then update weights using internal state



- Weight update (Adam)
  - States: 1 momenta, 1 variance
  - (reading and updating momenta/variance/parameters)

One iteration: forward pass; backward pass(weight+activation gradients); weight update

Reason of distributed training: large models, large datasets, system side(large memory size)

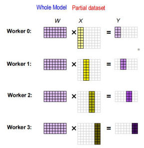
Solution: scale out computing.

Data parallelism

Model: has a copy of entire neural network model

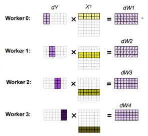
Dataset: responsible for compute of a portion of data(training minibatch)

Data Parallel: Forward Pass



- Forward pass:
  - Computes output activations for its portion of minibatch
  - No communication is needed
- X : input activations
- W : model
- Y : output activations

Data Parallel: Backward Pass



- Backward pass:
  - Computes activation gradients for its portion of minibatch
  - Computes contribution to the weight gradient based on its portion of minibatch
    - All workers' contributions must be summed before weight update
- X : input activations
- W : model
- Y : output activations

Weight update:each of N workers accumulates gradients; each worker updates its model

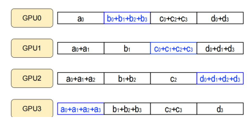
AllReduce implementation choices

Ring AllReduce:for any topology that contains a 1D torus(ring),each worker communicates with 2 neighbors, 2(n-1)steps, worker send/receive 1/n of all bytes, each worker need CPU&GPU.

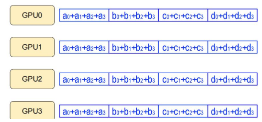
Two stages: reduce_scatter:N-1 rounds, M/N data per round;

allgather: n-1 rounds, M/N data per round.N:number of GPU;M:data size

"Ring" AllReduce: Reduce_scatter iter. 3



"Ring" AllReduce: Allgather iter. 3



In-switch allreduce:each worker communicate with the switch.only one step, a worker sends/receives N of all bytes.

Challenge of data parallel:

Strong scaling (increase the number of workers, keep minibatch size constant)

- Certain layers require minimum minibatch sizes to properly operate
- Maybe lower GPU utilization

Weak scaling (increase the number of workers, increase minibatch size)

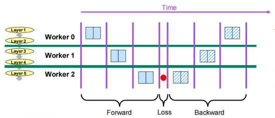
- Training networks with large minibatches requires hyper-parameter adjustment
  - Learning rate schedule, BN decay, …
- Often increase the amount of work required to reach the same model accuracy

Model parallelism: pipeline & tensor parallelism

Inter-layer parallel(aka pipeline parallel): worker responsible for its portion of the layers

Inter-layer parallel(aka tensor parallel):worker responsible for its portion of each layer

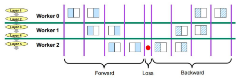
Pipeline Parallel Training



- Idle bubbles:
  - 67%: 12/18 step-slots
- For *N* workers:
  - $(N - 1)/N$ idle slots

Pipeline Parallel Training: GPipe

Key Idea: Subminibatches



- 2 subminibatches
  - 2x more steps
  - Each step is ½ compute
- Idle bubbles: 50%
  - 12/24 steps-slots

***N* workers, *K* subminibatches:**

- $2(N + K - 1)$ steps for fwd/bwd
- Total step-slots: $2N(N + K - 1)$
- Idle step-slots: $2N(N - 1)$
- Fraction of idle slots: $(N - 1)/(N + K - 1)$

As *N* grows:

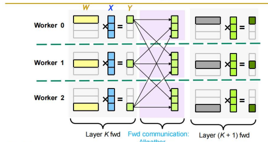
- $K = N \rightarrow 50\%$ idle slots
- $K = 4N \rightarrow 20\%$ idle slots

Challenges: load balancing workload across workers is difficult.lots of computation to hide communication. Idle slots reduce scaling efficiency.

Tensor parallel

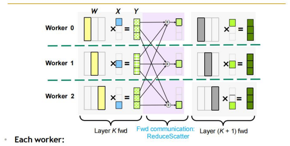
Two variants: row-wise partitioning, column-wise partitioning

Row-wise Partitioning: Allgather between Layers



- Each worker:
  - W: Has a portion of weight rows
  - X: All of input activations X
  - Y: Computes a portion of output activations

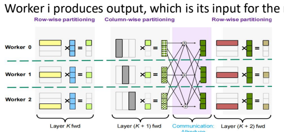
Column-wise Partitioning: ReduceScatter between Layers



- Each worker:
  - W: Has a portion of weight rows
  - X: All of input activations X
  - Y: Computes a portion of output activations

No communication is needed for two matrices

Worker i produces output, which is its input for the next layer.



Row/col-wise in forward becomes col/row-wise in backward.

Row-wise:fwd(allgather),bwd(reduce_scatter), col-wise:fwd(reduce_scatter),bwd(allgather)

Attention/MLP: col-wise+row--wise

Communication pattern

Data Parallel:

- Allreduce of weights
- Can be overlapped with computation

Pipeline Parallel:

- Point-wise communication of activations and activation gradients
- Hard to overlap with computation
- Hard to load-balance

Tensor Parallel:

- Allgather, Reduce_scatter of activations and activation gradients
- Allreduce if row-wise and col-wise partitioning is alternated
- Hard to overlap with computation

Zero

Zero redundancy optimizer:each GPU stores a subset of optimizer state,rather than whole state.

Scale-out enables these neural networks to be trained

Success requires many optimized components:

- Hardware:
  - Fast accelerators for DL
  - High-bandwidth, low-latency interconnects
    - Topologies matter (must match communication patterns)
    - Network switches with match capabilities free up DL accelerators to do compute
    - SmartNIC for offloaded compression/decompression
- Software:
  - Math libraries (CUDNN, CUBLAS, MKL, CANN …)
  - Collective communication libraries (NCCL, Horovod, …)
  - Training frameworks (MindSpore, PyTorch, TensorFlow, HugeCTR, …)
- Proper choice of parallelism (manual, MeshTensorFlow, Gshard, ZeRO)